Proyecto Integrador:

Diseño e implementación de un analizador sintáctico para *MiniHTML*

Modalidad: Grupal (hasta 4 integrantes)

Contenidos: Evaluación de Gramáticas Libres de Contexto y Autómatas a Pila

Implicancia: La aprobación de este proyecto reemplaza el segundo parcial para alcanzar la regularidad en la materia. La aprobación de este proyecto reduce la nota mínima requerida en los parciales a 6 para obtener a promoción directa.

1. Introducción y contexto

En el desarrollo de software moderno, la validación de estructuras de datos y lenguajes de marcado (como HTML, XML o JSON) es una tarea fundamental. Un error común en documentos HTML es el mal anidamiento de etiquetas (por ejemplo, Texto), el cual viola la estructura jerárquica del lenguaje.

El objetivo de este proyecto es aplicar los conceptos formales de la materia para diseñar, modelar e implementar un **analizador sintáctico (*parser*)** capaz de validar documentos en **MiniHTML**: un subconjunto de HTML que requiere memoria LIFO (pila) para verificar el balanceo y anidamiento estricto de sus etiquetas.

2. Definición del Lenguaje (*MiniHTML*)

El lenguaje *MiniHTML* reconoce cadenas compuestas por:

- **Etiquetas de apertura:** , ,
- **Etiquetas de cierre:** , ,
- **Texto plano:** representado por el token genérico TEXTO.

  Esto implica que las cadenas usan el alfabeto de 7 caracteres

$$\Sigma = \{< b >, < i >, < p >, <\backslash b >, <\backslash i >, <\backslash p >, TEXTO\}$$

Reglas de validez sintáctica:

1. Todo documento puede contener texto plano, etiquetas anidadas o estar vacío.

2. Toda etiqueta de apertura debe cerrarse con su correspondiente etiqueta del mismo tipo.
3. Las etiquetas deben cerrarse en orden estricto LIFO (Last-In, First-Out).

   Ejemplo válido:

   El documento :

   Hola mundo estudiantil

   Que se interpreta como

   TEXTO TEXTO TEXTO

   Tiene etiquetas correctamente anidadas

   Ejemplo inválido:

   El documento:

   Error de anidamiento

   Que se interpreta como

   TEXTO

   Tiene las etiquetas de cierre en orden incorrecto.

3. Consignas del proyecto

El proyecto se divide en tres fases correlativas que conectan la teoría matemática con la implementación en código:

Fase 1: Especificación formal de una gramática libre de contexto (G)

1. **Definición de la gramática:** Especificar formalmente $G = (\mathrm{V}, \Sigma, S, P)$. Identificar claramente las variables no terminales V y los símbolos terminales Σ, usar el alfabeto indicado anteriormente y las reglas de producción P.
2. **Análisis de la gramática:** Mostrar el árbol de derivación sintáctica para las cadenas:

$$\alpha = \text{<p> TEXTO TEXTO </p>}$$

$$\beta = \text{<p> TEXTO TEXTO <i> TEXTO </i> </p>}$$

Fase 2: Diseño formal del autómata a pila (M)

1. **Definición del APD o APND:** Especificar el autómata a pila determinista o no determinista por vaciado de pila o estado final $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$.
2. **Función de transición (δ):** Presentar la tabla y el diagrama de transiciones indicando claramente el manejo de la pila.
3. **Trazas de configuración instantánea:** Presentar la secuencia paso a paso $(q_0, w, Z_0) \vdash^* (q, \lambda, \gamma)$ para las cadenas

$$\alpha = \text{<p> TEXTO TEXTO </p>}$$
$$\beta = \text{<p> TEXTO TEXTO <i> TEXTO </i> </p>}$$
$$\gamma = \text{<p> TEXTO TEXTO </p> <i> TEXTO </i>}$$

Fase 3: Implementación del analizador sintáctico (*Parser*)

Implementar el reconocedor ejecutable en el lenguaje de programación de preferencia del grupo (Python, C++, Java, JavaScript, C#). El reconocedor debe recibir una cadena de entrada y determinar si es válida o no en MiniHTML.

1. **Estrategia de implementación:** se puede implementar con alguna de estas dos opciones:
   - **Opción A (Analizador sintáctico a partir de G):** Puede definir una función o procedimiento para cada variable no terminal de la gramática (por ejemplo: validar_documento(), validar_contenido(), validar_elemento()).Utilice la llamada recursiva entre estas funciones para controlar el balanceo de etiquetas de apertura y cierre a medida que el programa lee la entrada. Para cada cadena de entrada, el programa debe responder si la cadena pertenece o no al lenguaje ($w \in L(G)$).
   - **Opción B (Simulador de AP):** El programa debe implementar la lógica de procesamiento del **Autómata a Pila** diseñado en la fase 2. Debe procesar la secuencia de entrada de forma secuencial y utilizar una estructura de datos tipo **Pila (*Stack*)** para gestionar el anidamiento y balanceo de las etiquetas. Para la cadena de entrada, el programa debe responder si la cadena pertenece o no al lenguaje ($w \in L(M)$).
2. **Manejo de Errores:** En caso de recibir una cadena inválida, el programa debe emitir un mensaje descriptivo indicando el motivo del fallo sintáctico.

4. Presentación

Deberá presentar este proyecto en formato escrito digital y oral.

1. **Informe Técnico (PDF):** Deberá contener una carátula con el nombre de los integrantes del grupo, el desarrollo de la fase 1 (Gramática) y fase 2 (Autómata a Pila). Además, deberá explicar brevemente por qué un Autómata Finito no es capaz de validar este lenguaje de anidamiento y por qué se requiere la pila.

   No transcriba las consignas en el informe. Deberá cargar este informe en el aula virtual de la materia.

2. **Evaluación oral y testeo en vivo:**

Se requerirá participación de todos los integrantes del grupo en una entrevista de evaluación oral. Deberá concretar una entrevista con uno de los docentes de la materia. Durante la entrevista se llevará a cabo una prueba de funcionamiento del programa en vivo: el docente ingresará cadenas de prueba en tiempo real (tanto válidas como inválidas) y el programa deberá clasificar correctamente cada cadena ingresada ($w \in L(G)$ o $w \notin L(G)$) y, en caso de error sintáctico, reportar adecuadamente el fallo. Además, se solicitará al grupo realizar una modificación menor en el momento, sobre el código fuente (por ejemplo, incorporar una nueva etiqueta al lenguaje, agregar una regla de producción o ajustar una validación).

El docente podrá realizar preguntas individuales sobre la correspondencia entre la Gramática libre de contexto (G), el Autómata a pila (M) y la estructura del código presentado.

5. Rúbrica de evaluación

Criterio	Excelente (100%)	Satisfactorio (70%)	Requiere Revisión (40%)	Ponderación
Modelado Formal (G y M)	Gramática y Autómata a Pila definidos correctamente con sus 4 y 7-tuplas. Tabla δ clara y trazas sin errores.	La gramática o el autómata presentan imprecisiones menores en símbolos o transiciones.	Presenta errores conceptuales graves en el manejo de la pila o faltan las trazas; o las reglas de G tienen falencias graves	**35%**
Implementación del Parser	Código limpio, modular y funcional. Mapea fielmente el modelo formal (G o M) y maneja errores adecuadamente.	El parser funciona en la mayoría de los casos, pero falla en casos borde o carece de un	El código no compila/ejecuta, o no utiliza la lógica de gramática/pila requerida.	**35%**

Criterio	Excelente (100%)	Satisfactorio (70%)	Requiere Revisión (40%)	Ponderación
		buen reporte de errores.		
Defensa Oral y Demostración en Vivo	Supera exitosamente el testeo con las cadenas del docente. El grupo realiza la modificación solicitada en vivo de manera rápida y correcta, demostrando dominio del código. Responden con solvencia técnica, justificando la relación código-teoría.	Responde correctamente ante la mayoría de las cadenas. El alumno justifica la mayor parte de las decisiones de diseño con ayuda menor.	El programa falla ante casos simples o errores sintácticos simples. El alumno no puede explicar el funcionamiento del código.	**30%**